

函数用途与设计思路

本文档按照项目模块，对主要函数的用途和设计思路进行说明。项目整体采用分层设计：StationManager 负责站点数据管理，MetroGraph 负责地铁图结构构建，PathFinder 负责路径规划算法，MenuSystem 负责用户交互，main() 负责程序初始化与模块整合。

1. 公共工具函数

函数	用途	设计思路
trimCsvField()	去除 CSV 字段前后的空格、换行符等字符。	CSV 文件最后一列可能带有 \r 或空格，如果不处理，可能导致 "开启"、"关闭" 等字符串判断失败。因此在读取字段后统一清理，提高数据解析的稳定性。
resolveDataPath()	自动寻找数据文件路径。	程序运行目录可能不同，例如 Visual Studio 调试目录、可执行文件目录或项目根目录。该函数会从当前目录及上级目录中查找数据文件，避免因相对路径不同导致 data 文件打不开。

2. 程序入口

函数	用途	设计思路
main()	程序入口。	程序启动后，先加载站点数据，再根据边数据构建地铁图，随后创建 PathFinder 和 MenuSystem 对象，最后进入菜单循环，实现各模块之间的整合调用。

3. 站点管理模块 StationManager

函数	用途	设计思路
parseStatusText()	将字符串状态转换为枚举状态。	CSV 文件中站点状态使用 "开启"、"关闭" 表示，程序内部使用 StationStatus::Open 和 StationStatus::Closed 表示。该函数负责两种表示方式之间的转换。
toStatusText()	将枚举状态转换为字符串。	保存 CSV 文件时，需要将程序内部的枚举状态重新转换为 "开启" 或 "关闭"，保证文件内容可读。
rebuildIndex()	重建站点 id 到数组下标的索引。	使用 unordered_map 建立 id -> index 映射，使后续通过站点 id 查询站点时不需要遍历整个数组，提高查询效率。
loadFromCsv()	从 Station.csv 读取站点数据。	按行读取 CSV 文件，解析站点 id、站名、所属线路和状态，并存入 stations_。读取完成后调用 rebuildIndex() 建立快速查询索引。
saveToCsv()	将站点数据保存回 CSV 文件。	当站点状态发生修改后，将当前内存中的站点列表重新写入 Station.csv，保证修改结果能够持久保存。
resetFromInitCsv()	恢复站点初始状态。	从 Station_init.csv 重新加载初始站点状态，再保存到 Station.csv，用于恢复系统默认数据。
batchUpdateStatusFromCsv()	批量更新站点状态。	从 update_station_status.csv 中读取多个站点的状态修改信息，并逐个调用状态修改函数，适合模拟多个站点同时关闭或开启的情况。

函数	用途	设计思路
<code>getStationById()</code>	根据站点 id 查询站点。	通过 <code>idToIndex_</code> 快速定位站点，返回对应站点指针。如果站点不存在，则返回 <code>nullptr</code> 。
<code>setStationStatus()</code>	设置站点状态。	先根据 id 找到站点，再修改其状态。若站点不存在，则返回失败。
<code>setStationOpen()</code>	设置站点开启或关闭。	对 <code>setStationStatus()</code> 进行封装，使用布尔值表示开启或关闭，使调用时更直观。
<code>isStationOpen()</code>	判断站点是否开启。	路径规划时需要跳过关闭站点，因此提供统一接口判断站点当前是否可用。
<code>getClosedStations()</code>	获取所有关闭站点。	遍历站点列表，筛选状态为关闭的站点，用于“显示当前关闭站点”功能。
<code>getStationsByLine()</code>	获取某条线路的所有站点。	根据线路名称筛选站点，用于显示指定线路的站点信息。
<code>searchStationByName()</code>	根据关键字模糊查询站点。	使用 <code>std::string::find()</code> 判断站名是否包含用户输入的关键字，实现简单的模糊匹配。
<code>getAllStations()</code>	获取所有站点。	为建图、线路展示、换乘判断等模块提供完整站点数据。
<code>hasStation()</code>	判断某个站点 id 是否存在。	建图和路径规划时用于过滤无效站点，避免访问不存在的数据。

4. 地铁图模块 MetroGraph

函数	用途	设计思路
<code>buildFromCsv()</code>	从 <code>Edge.csv</code> 构建地铁图。	将每条站点连接关系转换为边，存入邻接表 <code>adj_</code> 中。邻接表适合表示地铁线路这种稀疏图，便于路径搜索时快速获取相邻站点。
<code>hasStation()</code>	判断图中是否存在某站点。	根据站点 id 判断其是否在邻接表范围内，用于路径规划中的合法性判断。
<code>vertexCount()</code>	返回图中顶点数量。	路径算法需要根据顶点数量初始化距离数组、前驱数组等辅助结构。
<code>neighbors()</code>	获取某站点的相邻边。	路径搜索时从当前站点扩展到相邻站点，该函数提供当前站点的所有可达边。
<code>countEdges()</code>	统计图中边的数量。	用于调试或显示建图结果，检查 <code>Edge.csv</code> 是否成功加载。

5. 路径规划模块 PathFinder

函数	用途	设计思路
<code>PathFinder()</code>	构造路径规划对象。	保存 <code>MetroGraph</code> 和 <code>StationManager</code> 的引用，使路径规划时既能访问图结构，也能判断站点状态。
<code>isTransferEdge()</code>	判断是否为换乘边。	如果边类型是 <code>Transfer</code> 或线路名为“换乘”，则认为该边表示站内换乘。
<code>countsAsTransfer()</code>	判断该边是否计入换乘次数。	起点处的换乘不计入换乘次数，其他换乘边才累计，避免起点线路选择被误算为换乘。

函数	用途	设计思路
makePathKey()	生成路径唯一标识。	将路径中的站点 id 拼接成字符串，用于多路径查询时判断路径是否重复。
containsStation()	判断路径中是否已包含某站点。	多路径搜索时用于避免形成环路，保证路径不会重复经过同一站点。
isStationAvailable()	判断站点是否可用。	站点必须存在于图中，并且状态为开启，路径规划才允许经过该站。
countTransfers()	统计路径换乘次数。	遍历路径中的相邻站点，根据边类型判断是否发生换乘，并累计换乘次数。
buildPathResult()	构造路径查询结果。	根据前驱数组还原完整路径，并统计总时间、途经站点数量、线路信息和换乘站点，最终封装为 PathResult。
shortestTimePath()	查询单条时间最短路径。	以总运行时间为主要权重进行最短路径搜索，适合用户希望尽快到达目的地的情况。
minTransferPath()	查询单条换乘次数最少路径。	优先比较换乘次数，若换乘次数相同，再比较总时间，适合用户希望减少换乘的情况。
kShortestTimePaths()	查询 3 条时间较短路径。	使用优先队列保存候选路径，按总时间从小到大扩展，并避免重复路径，为用户提供多条备选路线。
kMinTransferPaths()	查询 3 条换乘较少路径。	与多条时间最短路径类似，但排序优先级改为换乘次数，其次才比较总时间。
analyzeAffectedArea()	分析关闭站点影响范围。	根据关闭站点的相邻边，找出与关闭站点直接相连、可能受到影响的周边站点。

6. 菜单交互模块 MenuSystem

函数	用途	设计思路
MenuSystem()	构造菜单系统。	保存站点管理器、地铁图和路径规划器的引用，便于菜单函数调用各模块功能。
showMainMenu()	显示主菜单。	将系统主要功能分为站点管理、时间最短路径、换乘最少路径和退出系统。
showStationMenu()	显示站点管理菜单。	集中展示站点状态管理、线路查询、影响分析和站点查询功能。
showTimePathMenu()	显示时间最短路径菜单。	提供单条路径和 3 条路径两种查询方式。
showTransferPathMenu()	显示换乘最少路径菜单。	提供单条换乘最少路径和 3 条换乘较少路径查询。
run()	主菜单循环。	持续读取用户输入，根据菜单编号调用对应功能，直到用户选择退出系统。
handleStationMenu()	处理站点管理菜单。	根据用户选择调用批量更新、手工更新、线路显示、站点查询等功能。
handleTimePathMenu()	处理时间最短路径菜单。	根据用户选择调用单条或 3 条时间最短路径查询。
handleTransferPathMenu()	处理换乘最少路径菜单。	根据用户选择调用单条或 3 条换乘最少路径查询。
batchUpdateStationStatus()	批量更新站点状态界面。	调用 batchUpdateStatusFromCsv() 读取批量更新文件，并将结果保存到 CSV。

函数	用途	设计思路
manualUpdateStationStatus()	手工更新站点状态界面。	接收站点 id 和状态输入，调用站点管理模块修改站点状态。
resetStationStatus()	恢复初始状态界面。	调用 resetFromInitCsv()，将站点状态恢复为初始数据。
showClosedStations()	显示当前关闭站点。	调用 getClosedStations()，输出当前所有关闭站点。
lineNumberToName()	将线路编号转换为线路名称。	将数字 1-18 转换为 x 号线，并处理 19 对应浦江线、20 对应市域机场线。
showLineInfo()	显示线路站点信息。	根据线路编号查询站点，并以竖向线路图形式展示站点，同时显示换乘线路和关闭状态。
analyzeAffectedStations()	显示关闭站点影响分析。	先通过关键字选择站点，再调用 analyzeAffectedArea() 输出受影响站点。
searchStationInfo()	查询站点详细信息。	根据关键字模糊匹配站点，并显示站点 id、站点名称、所属线路和运营状态。
selectStationByKeyword()	根据关键字选择站点。	用于路径查询和站点分析。先模糊匹配站点，再让用户从匹配结果中选择具体站点。
queryShortestTimePath()	查询单条时间最短路径。	获取起点和终点，调用 shortestTimePath()，再输出路径结果。
queryKShortestTimePaths()	查询 3 条时间最短路径。	获取起点和终点，调用 kShortestTimePaths()，并逐条输出结果。
queryMinTransferPath()	查询单条换乘最少路径。	获取起点和终点，调用 minTransferPath()，输出换乘次数较少的路径。
queryKMinTransferPaths()	查询 3 条换乘较少路径。	获取起点和终点，调用 kMinTransferPaths()，输出多条备选路径。
printPathResult()	输出路径结果。	统一显示预计车程、途经站点数量、换乘次数、路径线路、站点路径和换乘站点。
waitForEnter()	暂停等待用户继续。	每个功能执行完成后等待用户按回车，避免运行结果一闪而过。

7. 总体设计说明

本项目将功能划分为数据管理、图结构、路径规划和菜单交互四个部分，各模块之间通过清晰的接口进行调用。

- StationManager 负责所有站点数据的读取、保存、查询和状态维护。
- MetroGraph 负责将地铁连接关系转换为邻接表图结构。
- PathFinder 基于图结构完成最短时间路径、最少换乘路径和多路径查询。
- MenuSystem 负责用户输入、菜单跳转和结果展示。
- main() 负责初始化数据和启动系统。

这种分层设计降低了模块之间的耦合度，使得数据读取、路径算法和界面展示可以相对独立地开发、调试和维护。